

EXERCICE 3 (6 points)

Cet exercice porte sur les arbres binaires, les files et la programmation orientée objet.

Cet exercice comporte deux parties indépendantes.

PARTIE 1

Une entreprise stocke les identifiants de ses clients dans un arbre binaire de recherche.

On rappelle qu'un arbre binaire est composé de nœuds, chacun des nœuds possédant éventuellement un sous-arbre gauche et éventuellement un sous-arbre droit.

La taille d'un arbre est le nombre de nœuds qu'il contient. Sa hauteur est le nombre de nœuds du plus long chemin qui joint le nœud racine à l'une des feuilles. On convient que la hauteur d'un arbre ne contenant qu'un nœud vaut 1 et celle de l'arbre vide vaut 0.

Dans cet arbre binaire de recherche, chaque nœud contient une valeur, ici une chaîne de caractères, qui est, avec l'ordre lexicographique (celui du dictionnaire) :

- strictement supérieure à toutes les valeurs des nœuds du sous-arbre gauche ;
- strictement inférieure à toutes les valeurs des nœuds du sous-arbre droit.

Ainsi les valeurs de cet arbre sont toutes distinctes.

On considère l'arbre binaire de recherche suivant :

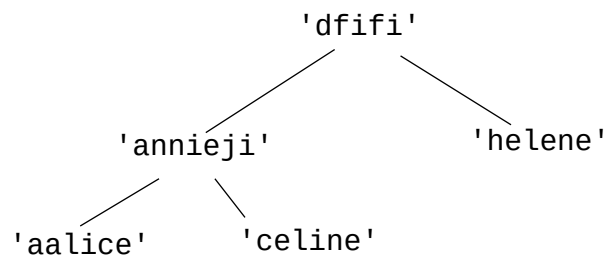


Figure 1. Arbre binaire de recherche.

1. Donner la taille et la hauteur de l'arbre binaire de recherche de la figure 1.
2. Recopier cet arbre après l'ajout des identifiants suivants : 'davidbg' et 'papicoeur' dans cet ordre.
3. On décide de parcourir cet arbre pour obtenir la liste des identifiants dans l'ordre lexicographique.

Recopier la lettre correspondant au parcours à utiliser parmi les propositions suivantes :

- A** - Parcours en largeur d'abord
- B** - Parcours en profondeur dans l'ordre préfixe
- C** - Parcours en profondeur dans l'ordre infixé
- D** - Parcours en profondeur dans l'ordre suffixé (ou postfixé)

4. Pour traiter informatiquement les arbres binaires, nous allons utiliser une classe ABR.

Un arbre binaire de recherche, nommé `abr` dispose des méthodes suivantes :

- `abr.est_vide()` : renvoie `True` si `abr` est vide et `False` sinon.
- `abr.racine()` : renvoie l'élément situé à la racine de `abr` si `abr` n'est pas vide et `None` sinon.
- `abr.sg()` : renvoie le sous-arbre gauche de `abr` s'il existe et `None` sinon.
- `abr.sd()` : renvoie le sous-arbre droit de `abr` s'il existe et `None` sinon.

On a commencé à écrire une méthode récursive `present` de la classe ABR, où le paramètre `identifiant` est une chaîne de caractères et qui retourne `True` si `identifiant` est dans l'arbre et `False` sinon.

```
1 def present(self, identifiant):
2     if self.est_vide():
3         return False
4     elif self.racine() == identifiant:
5         return ...
6     elif self.racine() < identifiant:
7         return self.sd(). ...
8     else:
9         return ...
```

Recopier et compléter les lignes 5, 7 et 9 de cette méthode.

PARTIE 2

On considère une structure de données file que l'on représentera par des éléments en ligne, l'élément à droite étant la tête de la file et l'élément à gauche étant la queue de la file.

On appellera `f1` la file suivante :

	'bac'	'nsi'	'2023'	'file'	
--	-------	-------	--------	--------	--

On suppose que les quatre fonctions suivantes ont été programmées préalablement en langage Python :

- `creer_file()` : renvoie une file vide ;
- `est_vide(f)` : renvoie `True` si la file `f` est vide et `False` sinon ;
- `enfiler(f, e)` : ajoute l'élément `e` à la queue de la file `f` ;
- `defiler(f)` : renvoie l'élément situé à la tête de la file `f` et le retire de la file.

5. Les trois questions suivantes sont indépendantes.

- a. Donner le résultat renvoyé après l'appel de la fonction `est_vide(f1)`.
- b. Représenter la file `f1` après l'exécution du code `defiler(f1)`.
- c. Représenter la file `f2` après l'exécution du code suivant :

```
1 f2 = creer_file()
2 liste = ['castor', 'python', 'poule']
3 for elt in liste:
4     enfiler(f2, elt)
```

6. Recopier et compléter les lignes 4, 6 et 7 de la fonction `longueur` qui prend en paramètre une file `f` et qui renvoie le nombre d'éléments qu'elle contient. Après un appel à la fonction, la file `f` doit retrouver son état d'origine.

```
1 def longueur(f):
2     resultat = 0
3     g = creer_file()
4     while ... :
5         elt = defiler(f)
6         resultat = ...
7         enfiler(... , ...)
8     while not(est_vide(g)):
9         enfiler(f, defiler(g))
10    return resultat
```

7. Un site impose à ses clients des critères sur leur mot de passe. Pour cela il utilise la fonction `est_valide` qui prend en paramètre une chaîne de caractères `mot` et qui retourne `True` si `mot` correspond aux critères et `False` sinon.

```
1 def est_valide(mot):
2     if len(mot) < 8:
3         return False
4     for c in mot:
5         if c in ['!', '#', '@', ';', ':']:
6             return True
7     return False
```

Parmi les mots de passe suivants, recopier celui qui sera validé par cette fonction.

A - 'best@'

B - 'paptap23'

C - '2!@59fgds'

8. Le tableau suivant montre, sur deux exemples, l'évolution d'une file f3 après l'exécution de l'instruction `ajouter_mot(f3, 'super')` :

	état initial de f3	état de f3 après l'instruction ajouter_mot(f3, 'super')										
Exemple 1	<table><tr><td></td><td>'bac'</td><td>'nsi'</td><td>'2023'</td><td></td></tr></table>		'bac'	'nsi'	'2023'		<table><tr><td></td><td>'super'</td><td>'bac'</td><td>'nsi'</td><td></td></tr></table>		'super'	'bac'	'nsi'	
	'bac'	'nsi'	'2023'									
	'super'	'bac'	'nsi'									
Exemple 2	<table><tr><td></td><td>'test'</td><td>'info'</td><td></td><td></td></tr></table>		'test'	'info'			<table><tr><td></td><td>'super'</td><td>'test'</td><td>'info'</td><td></td></tr></table>		'super'	'test'	'info'	
	'test'	'info'										
	'super'	'test'	'info'									

Écrire le code de cette fonction `ajouter_mot` qui prend en paramètres une file `f` (qui a au plus 3 éléments) et une chaîne de caractères valide `mdp`. Cette fonction met à jour la file de stockage `f` des mots de passe en y ajoutant `mdp` et en défilant, si nécessaire, pour avoir au maximum trois éléments dans cette file.

On pourra utiliser la fonction `Longueur` de la question 6.

9. Pour intensifier sa sécurité, le site stocke les trois derniers mots de passe dans une file et interdit au client lorsqu'il change son mot de passe d'utiliser l'un des mots de passe stockés dans cette file.

Recopier et compléter les lignes 7 et 8 de la fonction `mot_file` :

- qui prend en paramètres une file `f` et `mdp` de type chaîne de caractères ;
- qui renvoie `True` si le mot de passe est un élément de la file `f` et `False` sinon.

Après un appel à cette fonction, la file `f` doit retrouver son état d'origine.

```

1  def mot_file(f, mdp):
2      g = creer_file()
3      present = False
4      while not(est_vide(f)):
5          elt = defiler(f)
6          enfiler(g, elt)
7          if ...:
8              present = ...
9      while not(est_vide(g)):
10         enfiler(f, defiler(g))
11     return present

```

10. Écrire une fonction `modification` qui prend en paramètres une file `f` et une chaîne de caractères `nv_mdp`. Si le mot de passe `nv_mdp` répond bien aux **deux** exigences des questions 7 et 9, alors elle modifie la file des mots de passe stockés et renvoie `True`. Dans le cas contraire, elle renvoie `False`.

On pourra utiliser les fonctions `mot_file`, `est_valide` et `ajouter_mot`.